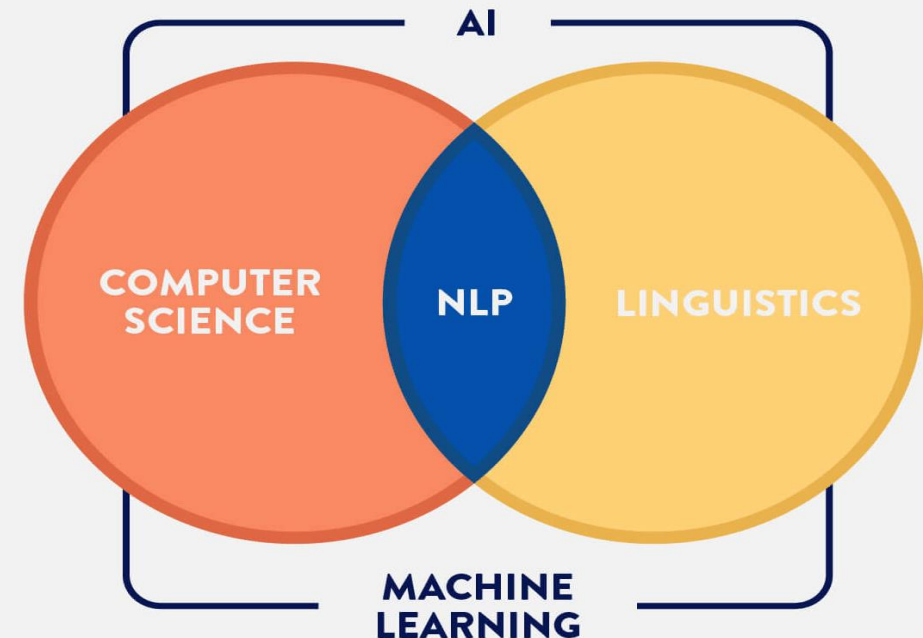
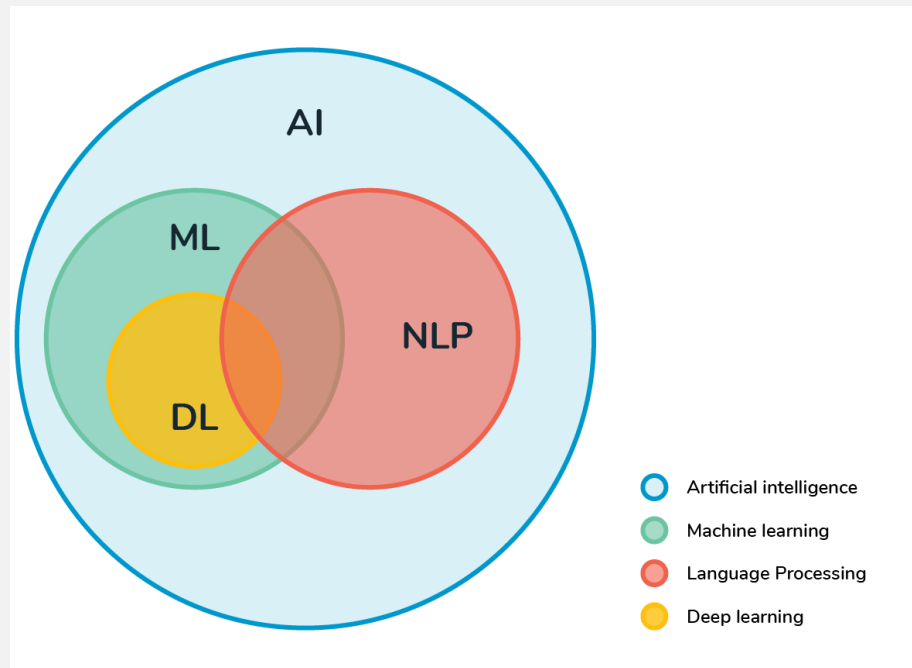


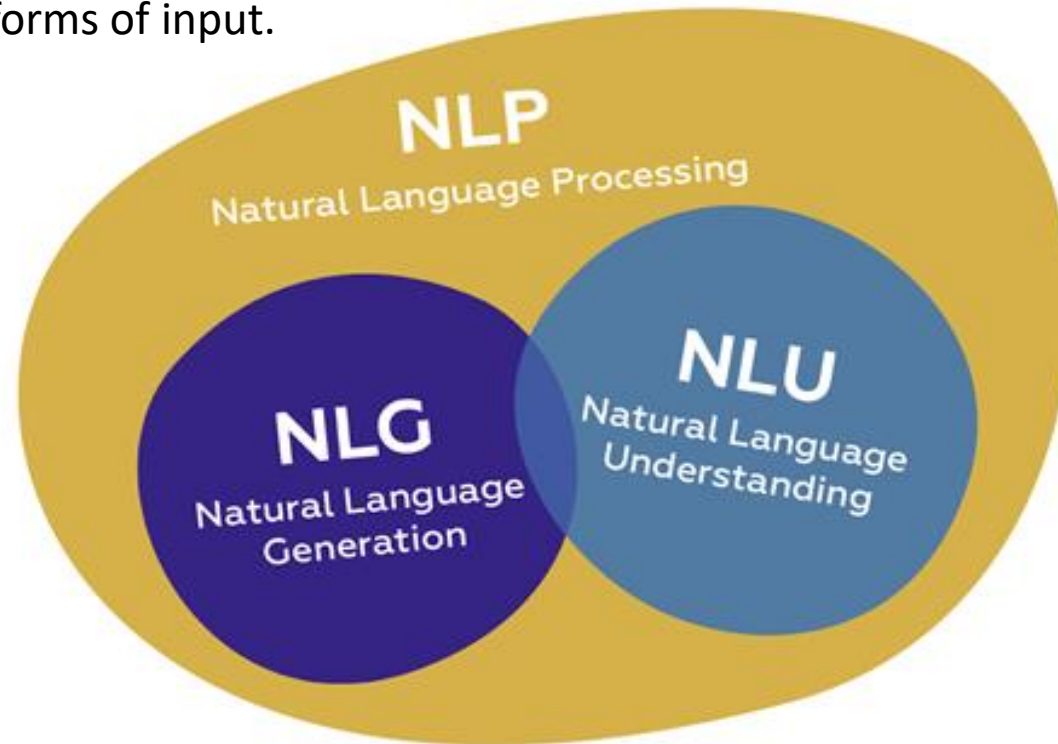
What is NLP?

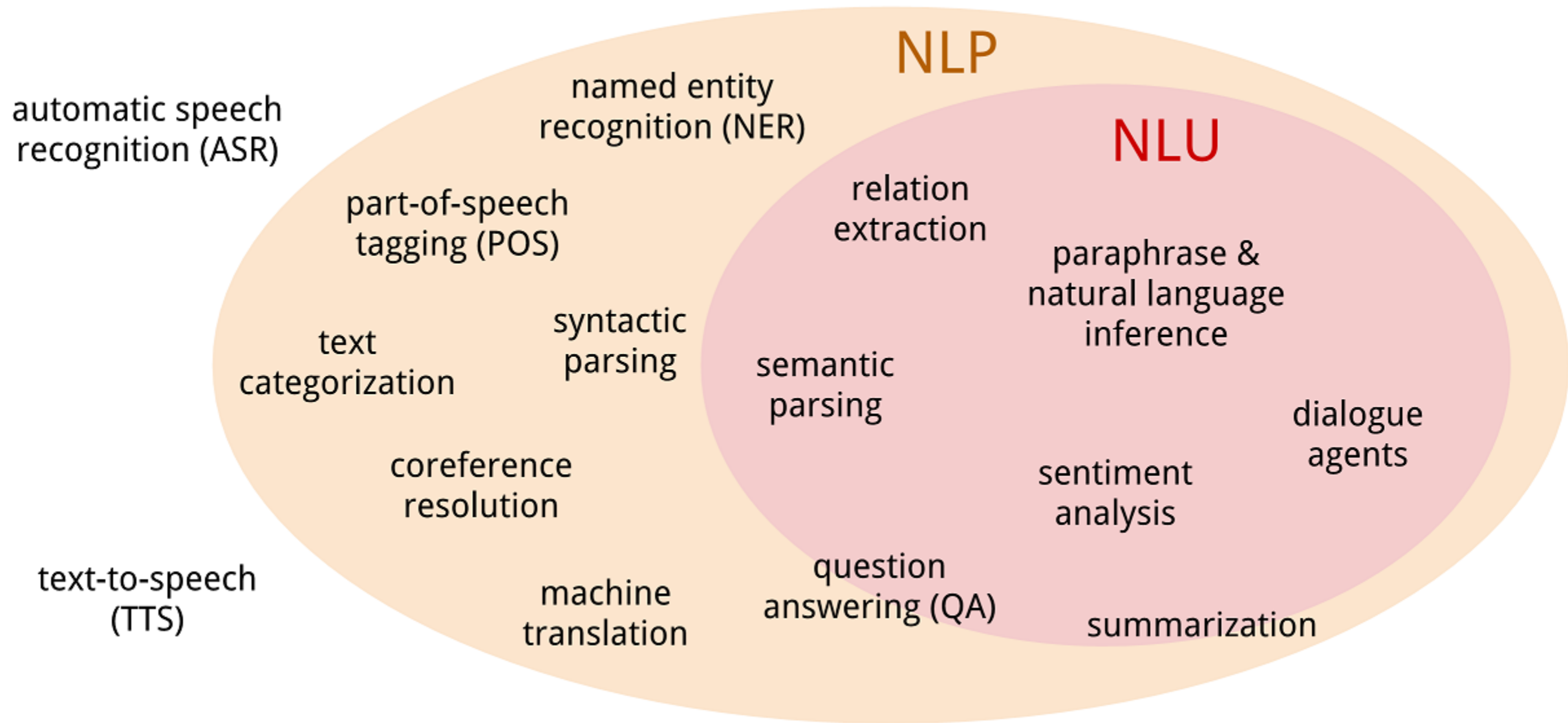
- Natural Language Processing (NLP) is a field within artificial intelligence (AI) and computational linguistics that focuses on the interaction between computers and human language.
- NLP enables machines to understand, interpret, generate, and respond to human language in a way that is both meaningful and useful.



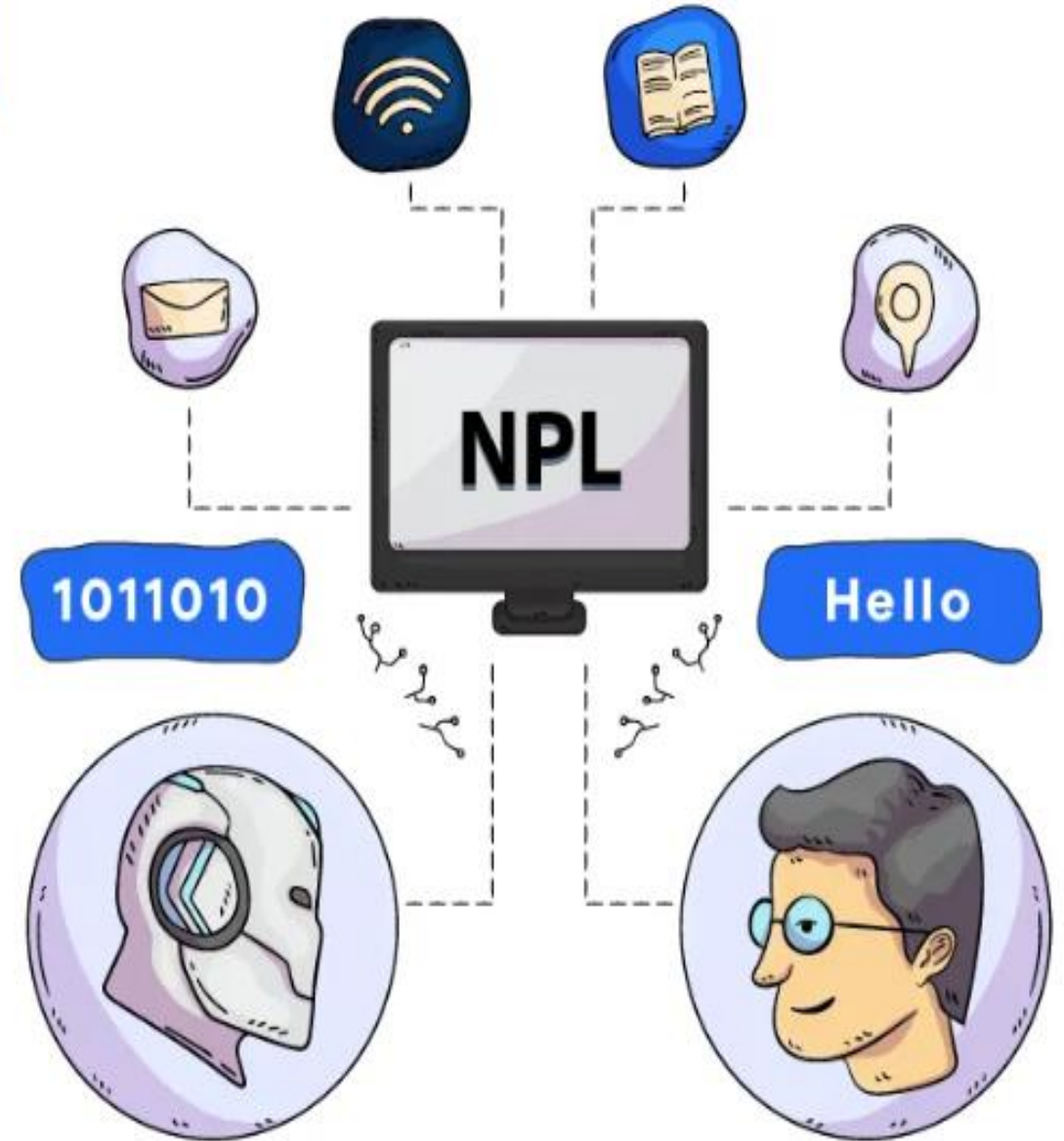
Components of NLP

- Natural Language Understanding (NLU) is a subfield of NLP that focuses on the machine's ability to comprehend and interpret human language. It involves extracting meaningful information and understanding the context, intent, and semantics of the text.
- Natural Language Generation (NLG) is a subfield of NLP that focuses on creating coherent and contextually appropriate text based on input data or prompts. It involves generating human-like language from structured data or other forms of input.





Why NLP Matters: Significance of Natural Language Processing



Applications of NLP

SENTIMENT ANALYSIS



POSITIVE

"Great service for an affordable price. We will definitely be booking again."



NEUTRAL

"Just booked two nights at this hotel."



NEGATIVE

"Horrible service. The room was dirty and unpleasant. Not worth the money."

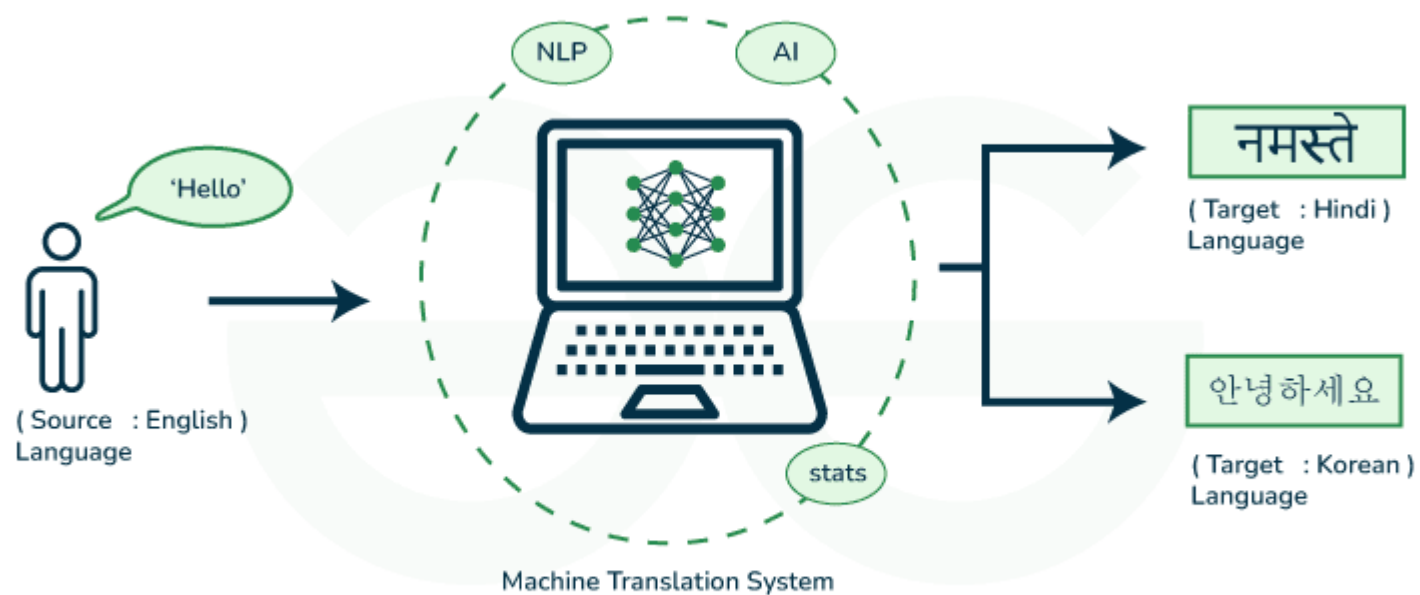
Given text, sentiment analysis classifies its emotional quality.

Applications of NLP

- **Toxicity classification** is a branch of sentiment analysis where the aim is not just to classify hostile intent but also to classify particular categories such as threats, insults, obscenities, and hatred towards certain identities.
- Toxicity classification models can be used to moderate and improve online conversations by [silencing offensive comments](#), [detecting hate speech](#), or [scanning documents for defamation](#).



Applications of NLP



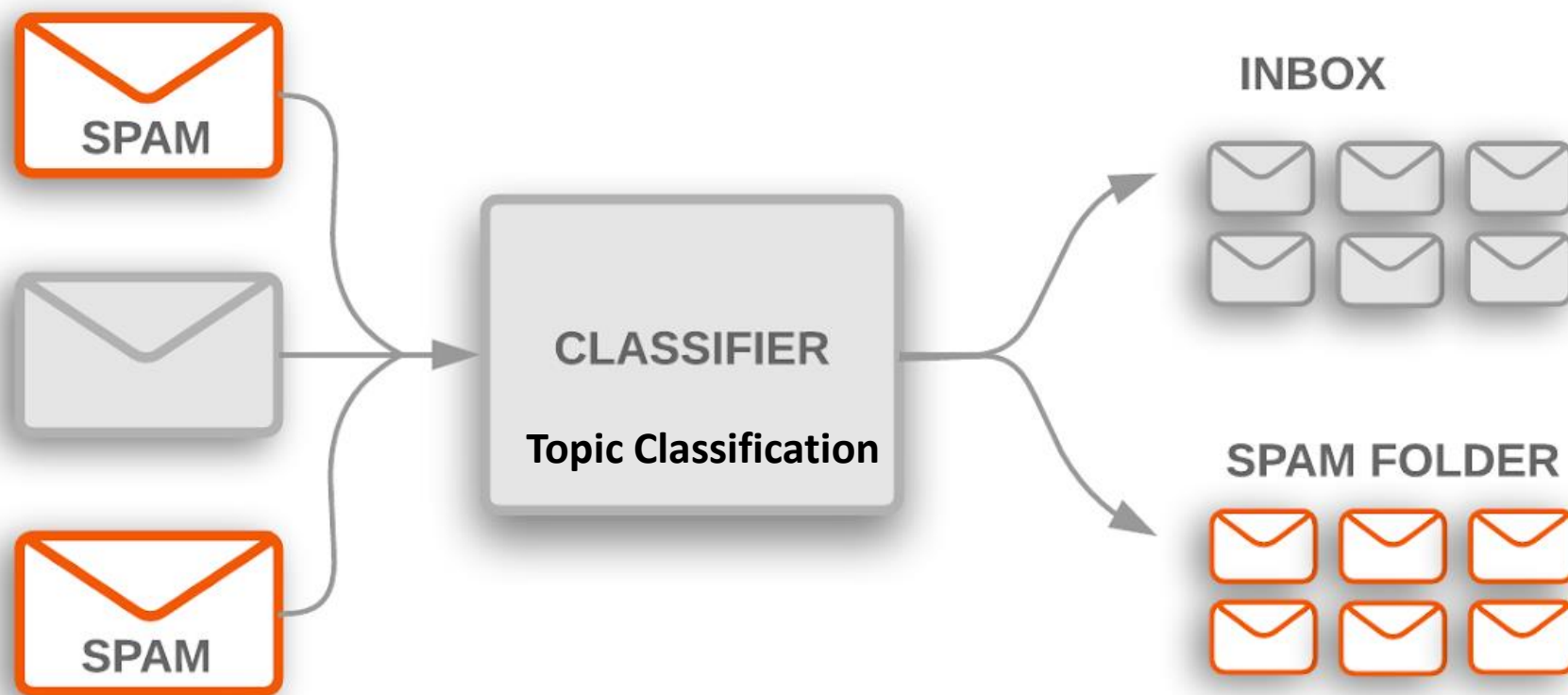
Applications of NLP

NAMED ENTITY RECOGNITION (NER) TAGGING

Andrew Yan-Tak Ng **PERSON** (**Chinese** **NORP** : 吳恩達; born **1976** **DATE**) is a **British** **NORP** -born **American** **NORP** computer scientist and technology entrepreneur focusing on machine learning and **AI** **GPE** .
Ng was a co-founder and head of **Google Brain** **ORG** and was the former chief scientist at **Baidu** **ORG** ,
building the company's **Artificial Intelligence Group** **ORG** into a team of **several thousand** **CARDINAL** people.

spaCy named entity recognition tagging of the first paragraph of Andrew Ng's Wikipedia page. "NORP" stands for nationalities or religious or political groups. Note that spaCy incorrectly labels "AI" as "GPE," for geopolitical entity.

Applications of NLP



Applications of NLP

Grammatical error correction [Grammarly](#)

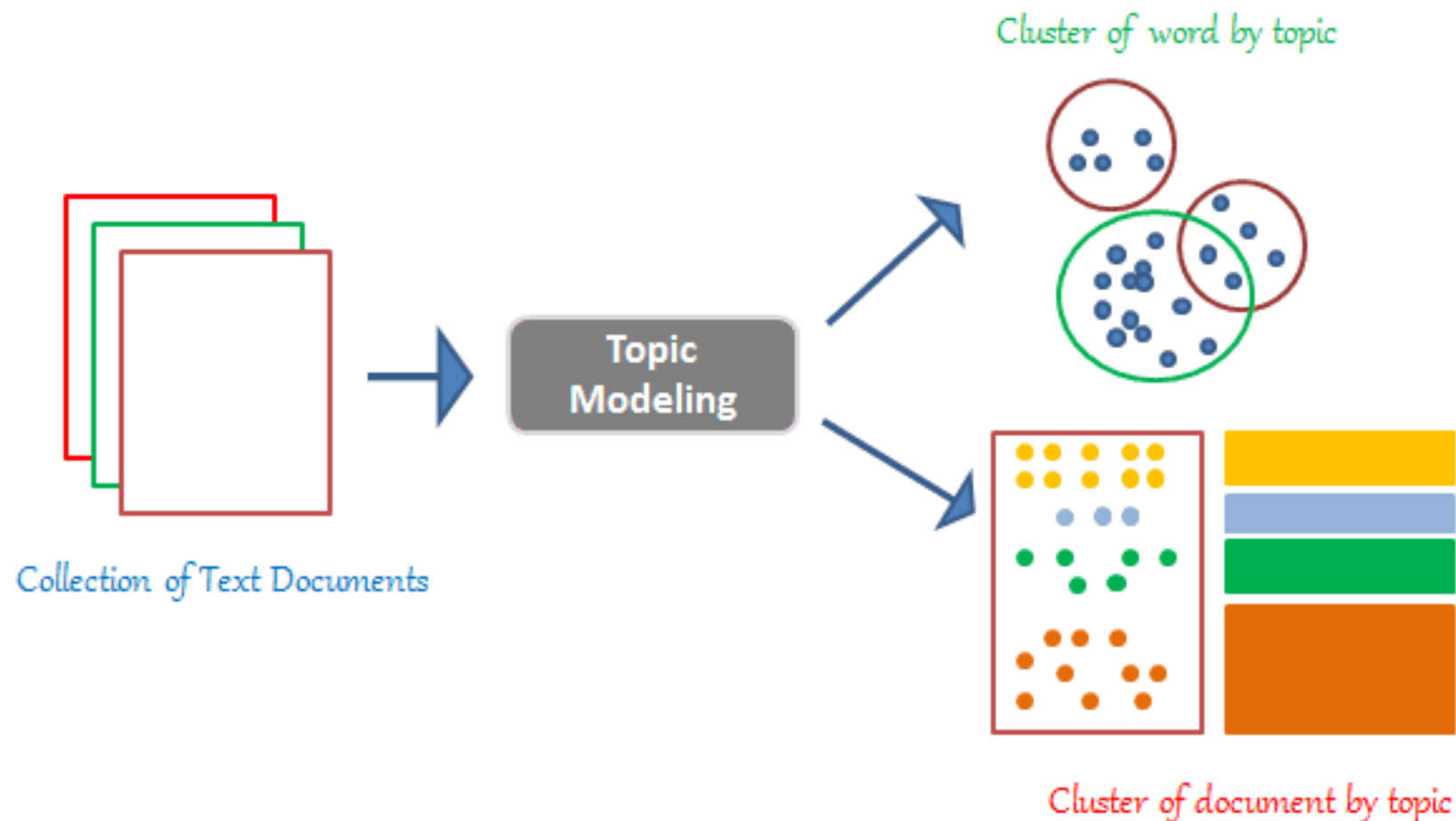
Input (Erroneous)

A important part of my life have been a people that stood by me.

Output (Corrected)

An important part of my life has been the people who stood by me.

Applications of NLP



Applications of NLP

Topic Modeling

Topic modeling is **unsupervised**, meaning it **doesn't require labeled training data**. Machine Learning algorithms simply look at the documents and group them into topics.

Topic modeling can find **hidden structures** in text data.

Topic modeling is more flexible than text classification since it **doesn't require a predefined set of categories**. Which can be time-consuming and expensive without adequate tools.

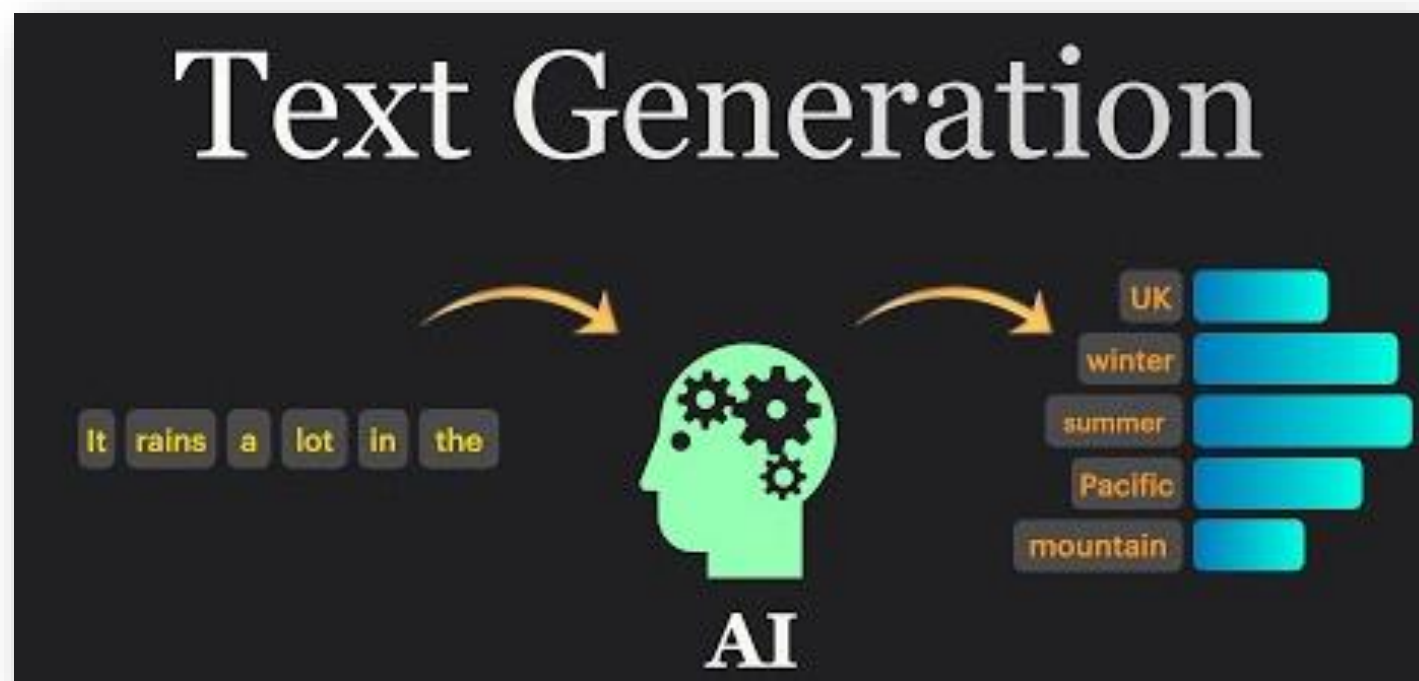
Text Classification

Text classification uses **supervised** Machine Learning techniques and **labeled training data** to learn how to classify text.

Text classification is more **focused on assigning labels to texts**.

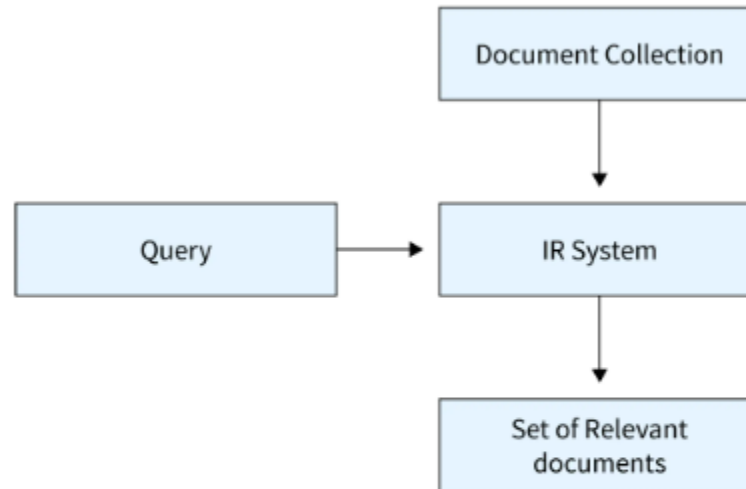
Text classification **requires pre-labeling** of the data,

Applications of NLP



Applications of NLP

Information retrieval finds the documents that are most relevant to a query.



Applications of NLP

Input Article

Marseille, France (CNN) The French prosecutor leading an investigation into the crash of Germanwings Flight 9525 insisted Wednesday that he was not aware of any video footage from on board the plane.

Marseille prosecutor Brice Robin told CNN that "so far no videos were used in the crash investigation." He added, "A person who has such a video needs to immediately give it to the investigators." Robin's comments follow claims by two magazines, German daily Bild and French Paris Match, of a cell phone video showing the harrowing final seconds from on board Germanwings Flight 9525 as it crashed into the French Alps. All 150 on board were killed. Paris Match and Bild reported that the video was recovered from a phone at the wreckage site. ...

Text Summarization Models

Abstractive summarization

Extractive summarization

Generated summary

Prosecutor : " So far no videos were used in the crash investigation "

Extractive summary

marseille prosecutor brice robin told cnn that " so far no videos were used in the crash investigation . " robin \s comments follow claims by two magazines , german daily bild and french paris match , of a cell phone video showing the harrowing final seconds from on board germanwings flight 9525 as it crashed into the french alps . paris match and bild reported that the video was recovered from a phone at the wreckage site .

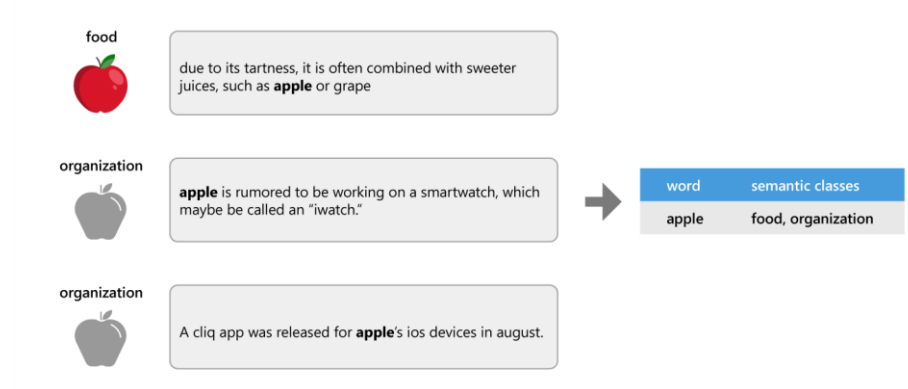
Applications of NLP

Question answering



NLP Challenges

- **Ambiguity**
- **Types:**
 - **Lexical Ambiguity:** **Words** with **multiple meanings** (e.g., "bank" can refer to a financial institution or the side of a river).
 - **Syntactic Ambiguity:** Sentences that can be **parsed** in multiple ways (e.g., "The man saw the woman with the telescope" can imply **different relationships** between the **man**, **woman**, and **telescope**).
 - **Semantic Ambiguity:** **Sentences** or **phrases** that can be interpreted in different ways based on context (e.g., "She is looking for a bank" could mean she's searching for a financial institution or the side of a river).
- **Challenges:**
 - Developing algorithms that can disambiguate based on context is complex.
 - Requires a deep understanding of both linguistic structure and real-world knowledge.



NLP Challenges

- **Context Understanding:** refers to the ability to comprehend and interpret information within its relevant circumstances, environment, and background
- **Types:**
 - **Long-Range Dependencies:** Capturing relationships between words or phrases that are **far apart** in a text (e.g., pronoun resolution).
 - **Conversational Context:** Maintaining context over **multiple turns** in a conversation or dialogue.
- **Challenges:**
 - Ensuring models remember and correctly interpret context from earlier parts of a conversation or document.
 - Difficult to handle with traditional models but improving with newer architectures like Transformers.

NLP Challenges

- Handling **Diverse** and **Noisy** Data
- Types:
 - **Social Media and Web Data:** Text from social media can be **informal**, full of **slang**, **abbreviations**, and **errors**.
 - **Multilingual Data:** Dealing with **multiple languages**, dialects, and language variations.
- **Challenges:**
 - Cleaning and preprocessing diverse data sources to ensure quality.
 - Building models that generalize well across different languages and informal text.

NLP Challenges

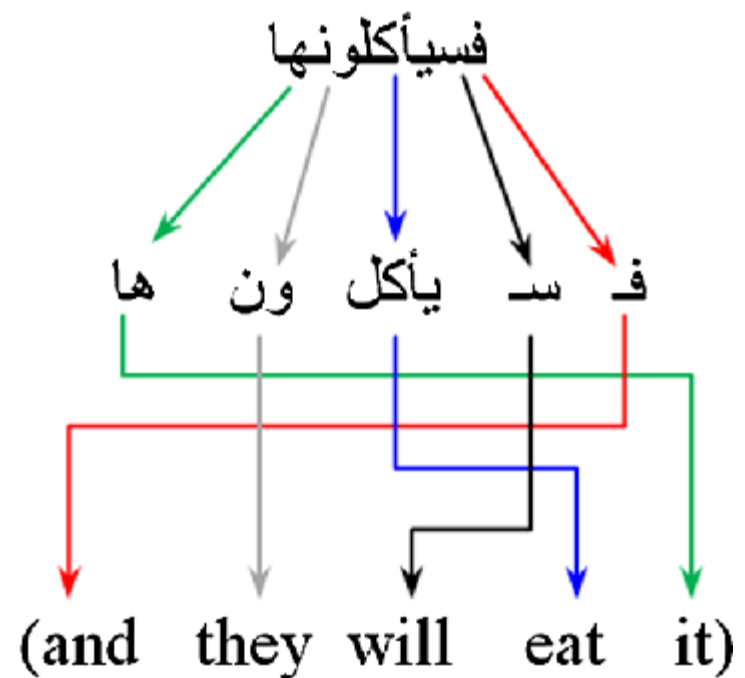
- Understanding **Implicit** Meaning
- Types:
 - **Sarcasm and Irony**: Identifying and correctly interpreting **sarcastic** or **ironic** statements.
 - **Cultural and Social Context**: Understanding references or implications specific to certain **cultures** or social groups.
- Challenges:
 - Designing models that can grasp subtle linguistic features and cultural context.
 - Training on diverse datasets that cover a wide range of social and cultural nuances.

NLP Challenges

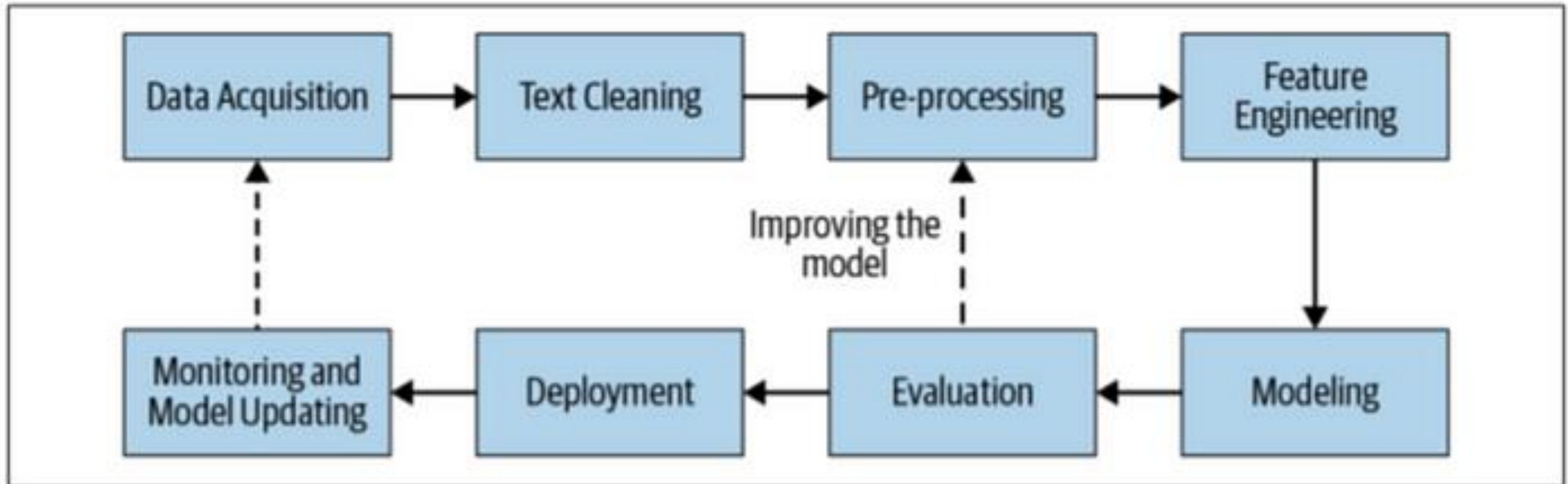
- **Low-Resource Languages**
- **Types:**
 - **Limited Data:** Many languages have **limited** available **training** data, making it challenging to build effective models.
 - **Under-Represented Dialects:** Some **dialects** or **regional variations** lack sufficient representation in data.
- **Challenges:**
 - Developing techniques to work effectively with small datasets or using transfer learning from high-resource languages.
 - Building models that respect and represent linguistic diversity adequately.

NLP Challenges

Necklace	عُقْدٌ
Decade	عِقْدٍ
Contract	عَقْدٌ
Held	عَقَدَ
Complicated	عَقَّ
Knots	عُقَدٌ

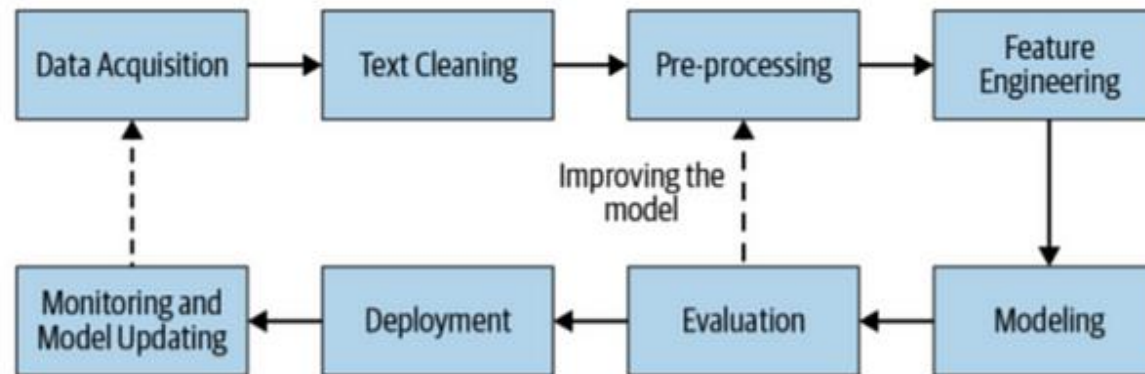


NLP Lifecycle



Text Cleaning

- Text cleaning is a **crucial step** in natural language processing (NLP) pipelines, as it **prepares raw text data** for further analysis and modeling.
- The goal is to **remove** or **correct parts** of the **text** that could introduce **noise** or **errors** into the **analysis**.



Text Cleaning

Text Lowercase

Data Science

Data science

DATA SCIENCE

data science

data science

Normal Text

LowerCase Text

Advantages of Lowercasing

- **Reduces Dimensionality:**
 - **Explanation:** By converting all text to lowercase, variations in capitalization (e.g., "Apple" vs. "apple") are normalized. This reduces the number of unique tokens or features, simplifying the model and reducing computational complexity.
 - **Benefit:** Helps in creating a more compact and manageable vocabulary, which can improve model efficiency and training speed.
- **Improves Consistency:**
 - **Explanation:** Lowercasing ensures that words are treated consistently regardless of how they are capitalized in the original text.
 - **Benefit:** Enhances the accuracy of text matching and similarity measurements, as variations in capitalization no longer affect the analysis.
- **Simplifies Text Processing:**
 - **Explanation:** Many NLP tasks, like text classification or sentiment analysis, do not require distinguishing between "Apple" and "apple."
 - **Benefit:** Simplifies the processing pipeline and reduces the need for handling case-specific variations, which can streamline text processing.
- **Reduces Sparsity:**
 - **Explanation:** Lowercasing helps in reducing the sparsity of feature matrices, where each unique capitalization variant would otherwise be treated as a distinct feature.
 - **Benefit:** Helps in improving model performance and generalization by reducing the number of features in the feature space.

Disadvantages of Lowercasing

- **Loss of Important Information:**
 - **Explanation:** Capitalization often conveys important information, such as proper nouns (e.g., "New York" vs. "new york") and sentence beginnings. Lowercasing can strip away this information.
 - **Drawback:** Can lead to loss of context or meaning, especially in cases where capitalization is used to convey specific entities or emphasis.
- **Inconsistent with Certain Use Cases:**
 - **Explanation:** In some applications, such as Named Entity Recognition (NER) or document classification, capitalization may carry crucial information that distinguishes entities or categories.
 - **Drawback:** Lowercasing might hinder the ability to recognize named entities or classify documents based on capitalized terms.
- **May Affect Model Performance:**
 - **Explanation:** For tasks where capitalization contributes to the understanding of the text (e.g., distinguishing between acronyms and regular words), lowercasing can degrade model performance.
 - **Drawback:** May negatively impact the accuracy of models that rely on case-sensitive features or where capitalization plays a role in distinguishing meanings.
- **Inconsistent with Some Languages:**
 - **Explanation:** Not all languages use capitalization in the same way. Some languages (e.g., Chinese) do not use capital letters, so lowercasing is less relevant.
 - **Drawback:** The practice may not be applicable or beneficial in all languages, and decisions about text preprocessing should consider language-specific conventions.

When to Use Lowercasing

- **Use Lowercasing When:**

- The task involves analyzing general text where capitalization does not convey essential meaning.
- The focus is on text classification, sentiment analysis, or other tasks where consistent token representation is more important than case distinctions.

- **Avoid Lowercasing When:**

- The task involves named entity recognition, where case distinctions are important.
- The text contains important case-specific information, such as acronyms, titles, or entities where capitalization carries meaning.

Text Cleaning

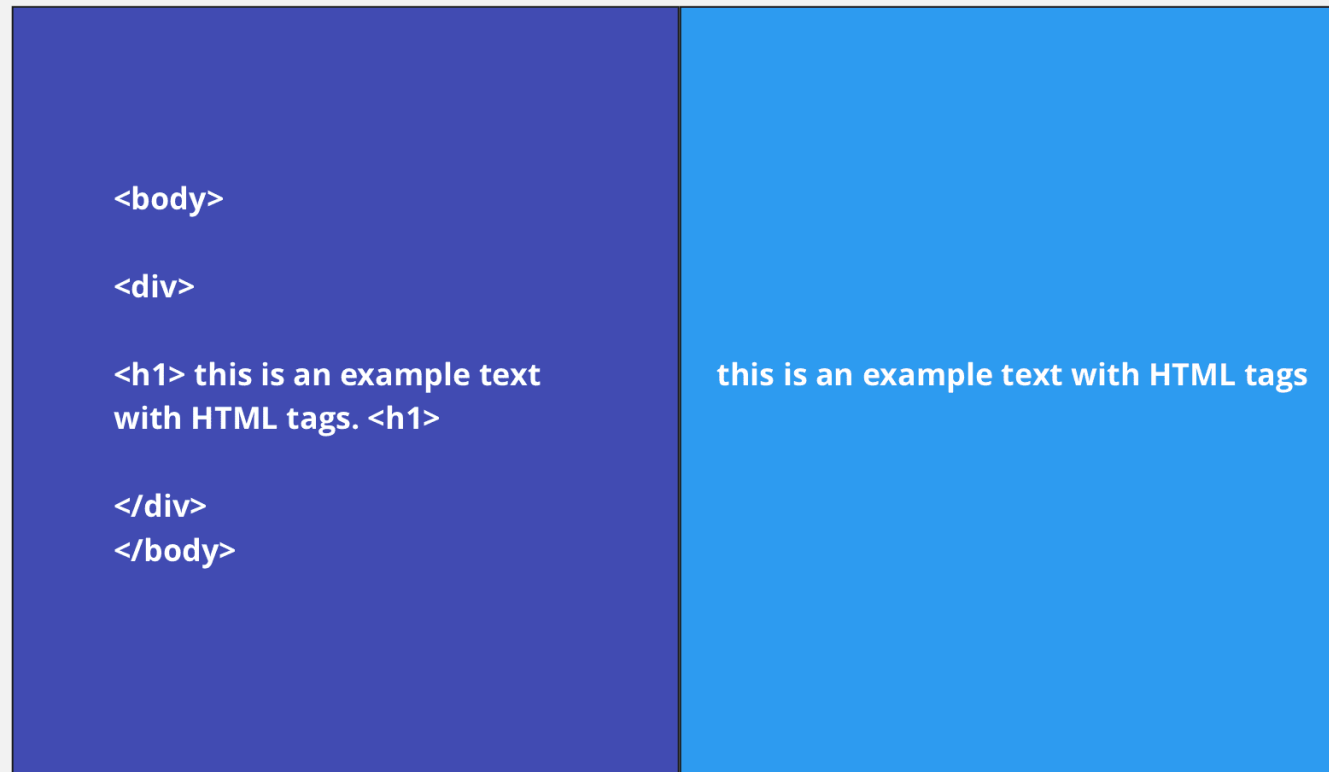
Spelling Correction

This is an example sentence
for spell **corecton**

This is an example sentence
for spell **correction**

Text Cleaning

HTML tags Removal



Text Cleaning

URL's Removal

This is an example text for URLs
like <http://google.com> &
<https://www.facebook.com/> etc.

This is an example text for URLs like & etc.

Text Cleaning

Removing Numbers

This is an example sentence
for removing numbers like **1,**
5,7, 4 ,77 etc.

This is an example sentence for
removing numbers like **, , ,** etc.

Text Cleaning

Numbers to Words

This is an example sentence for converting numbers to words like 1 to one, 5 to five, 74 to seventy-four, etc.

This is an example sentence for converting numbers to words like **one** to one, **five** to five, **seventy-four** to seventy-four, etc.

Text Cleaning

Spelling Correction

This is an example sentence
for spell **corecton**

This is an example sentence
for spell **correction**

Text Cleaning

Convert accented characters to ASCII characters

This is an example text with accented characters like dèèp lèarning ánd cömputer vísiön etc.

This is an example text with accented characters like deep learning and computer vision etc.

Text Cleaning

Converting chat conversion words to normal words

omg this is an example text for
chat conversation.

oh my god this is an example text for
chat conversation.

Text Cleaning

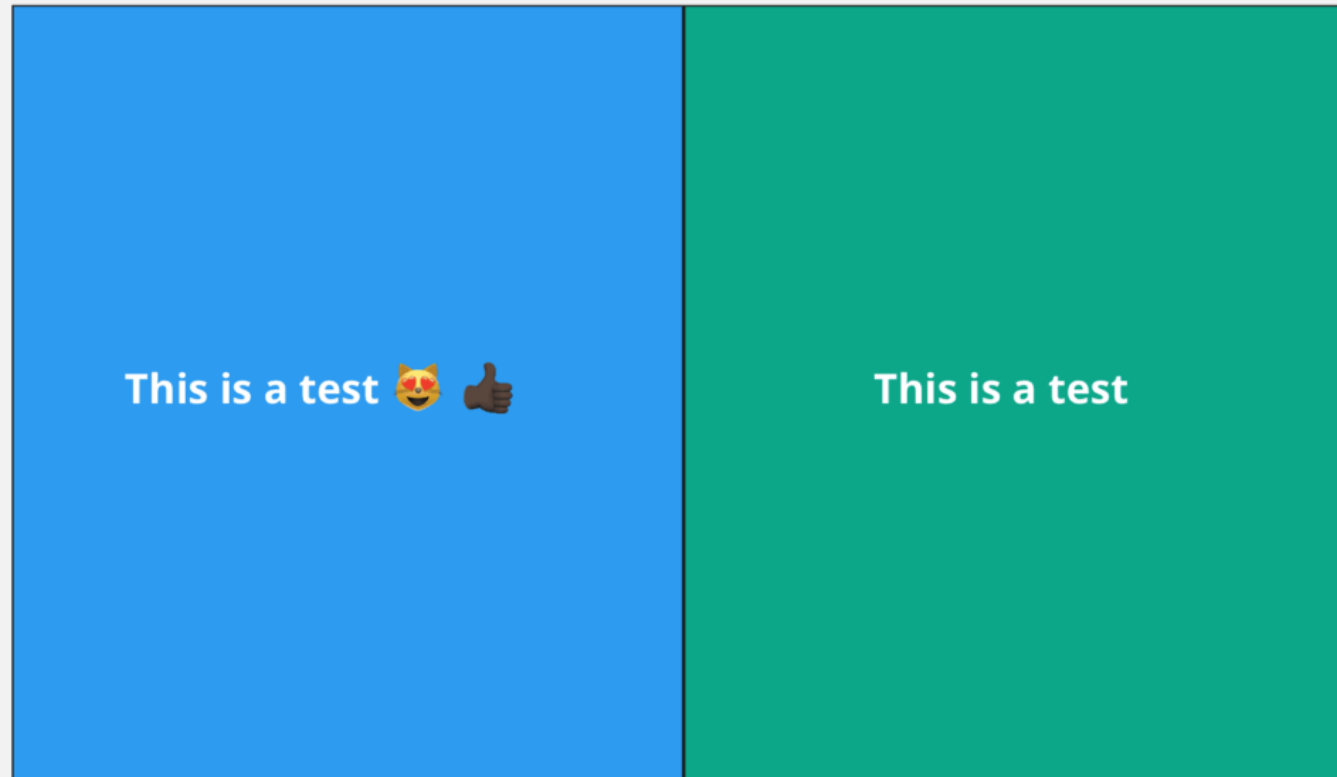
Expanding Contractions

Sometimes our mind **doesn't**
work properly.

Sometimes our mind
does not work properly.

Text Cleaning

Removal of Emojis



Text Cleaning

Emoticons Removal

Hello this is a sentence with
these 2 emoticons :-) & :-)

Hello this is a sentence with
these 2 emoticons &

Converting Emojis to words



Text Cleaning

Removing of Punctuations or Special Characters

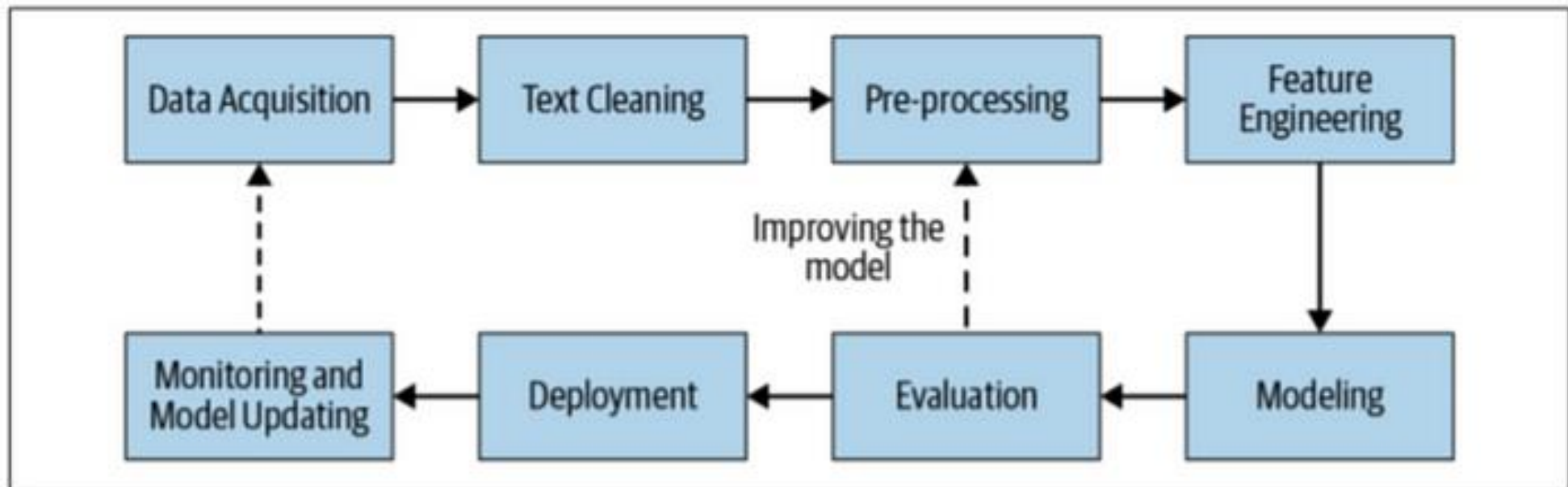
this is an example text for
punctuations like .?/*

this is an example text for
punctuations like

Self Study: Regex

Preprocessing

- Preprocessing is a crucial step in preparing text data for natural language processing (NLP) tasks. It helps to standardize and clean text to improve the performance of NLP models.



Tokenization

- **Definition:** The process of splitting text into individual tokens, such as words or phrases.
- **Techniques:**
 - **Character-based:** The simplest tokenization scheme is to feed each character individually to the model.
 - **Word Tokenization:** Breaking text into words.
 - **Sentence Tokenization:** Splitting text into sentences.
 - **Subword Tokenization:** Handling words that are not in the vocabulary by splitting into subword units (state-of-the-art transformer)

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

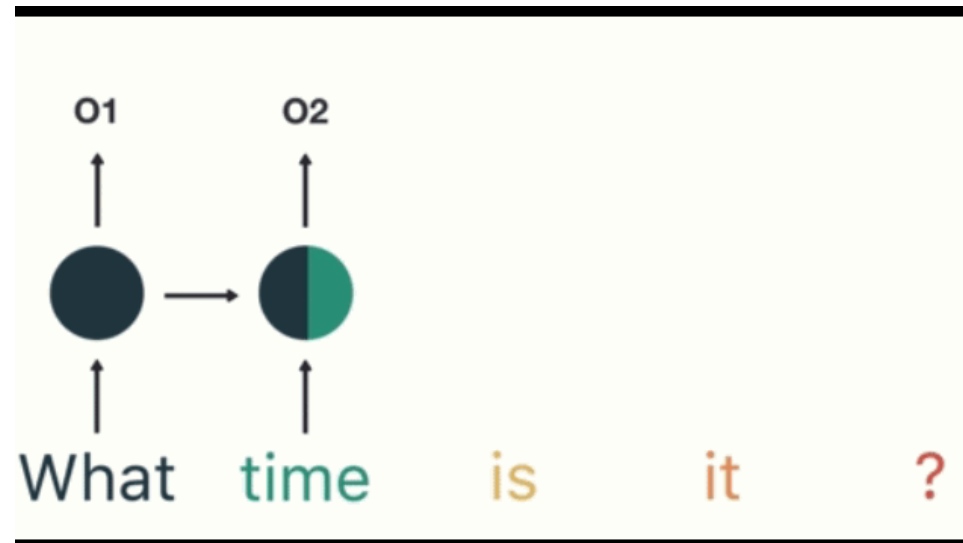
[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

Why do we tokenize?

- Tokens are the building blocks of text in the natural language.
- Therefore, most of the **preprocessing** and **modeling** happens at the **token level**.
- For example, removing stopwords, stemming, lemmatization and many other preprocessing steps happen at token levels
- Even neural network architectures process individual tokens to make sense of the document.

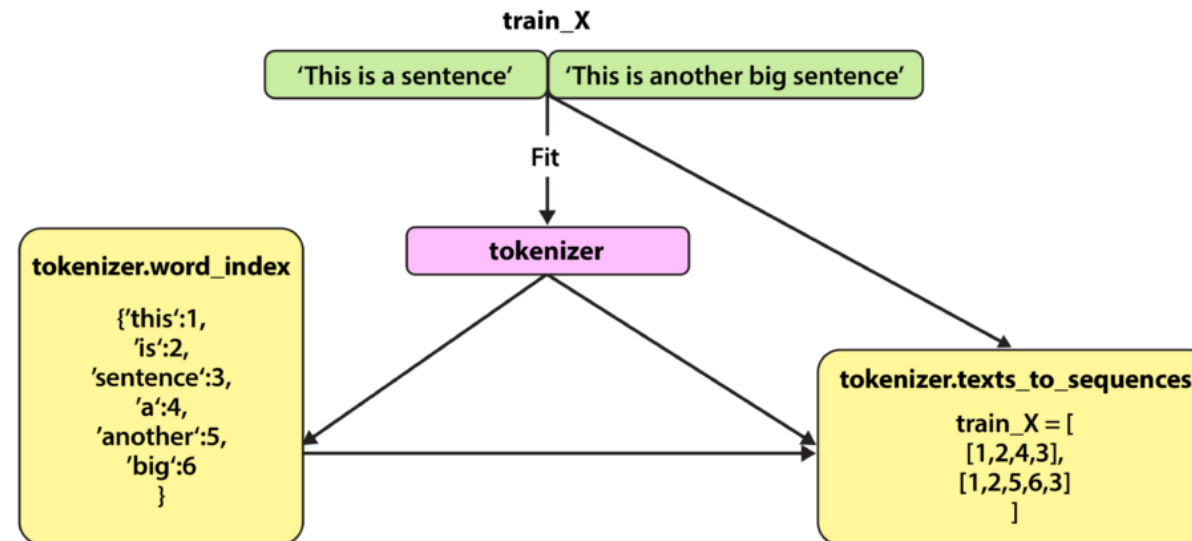


Challenges of Character tokenization

- We can see that character-level tokenization **ignores** any **structure** in the text and treats the whole **string** as a **stream of characters**.
- Although this helps deal with **misspellings** and **rare words**, the main drawback is that **linguistic structures** such as words need to be learned from the data.
- This requires significant **compute**, **memory**, and data. For this reason, character tokenization is **rarely** used in practice.

Word Tokenization

- Instead of splitting the text into characters, we can split it into **words** and map each word to an integer.
- Using words from the outset enables the model to **skip** the step of **learning words from characters**, and thereby **reduces** the **complexity** of the training process.



Given a corpus of documents, a tokenizer maps every word to an index. Then it can translate any document into a sequence of numbers.

Challenges of word tokenization

- **1. Handling Punctuation and Special Characters**
 - **Separation vs. Retention:** Deciding whether punctuation marks (e.g., commas, periods) should be treated as separate tokens or included with adjacent words can be complex. For example, "Hello, world!" could be tokenized as ["Hello", ",", "world", "!"] or ["Hello,", "world!"].
 - **Special Characters:** Special characters (e.g., hashtags, @mentions) in social media or URLs can complicate tokenization. Deciding how to handle these characters requires careful consideration.
- **2. Dealing with Multi-Word Expressions**
 - **Compound Words:** Words like "New York" or "machine learning" are often treated as single entities, but straightforward tokenization might split them into individual words.
 - **Named Entities:** Identifying and properly tokenizing named entities (e.g., "San Francisco", "United Nations") is challenging as they often appear as multi-word expressions.
- **Languages without clear boundaries.**
 - Some languages, like Chinese or Japanese, don't have clear spaces between words, making tokenization a more complex task. Determining where one word ends and another begins can be a significant challenge in such languages.
- **Handling Out-of-Vocabulary Words**
 - **Unknown Words:** Tokenization can struggle with words not seen during training, such as slang or newly coined terms. Handling these out-of-vocabulary words effectively is challenging.

Subword Tokenization

- The basic idea behind subword tokenization is to **combine** the best **aspects** of **character** and **word** tokenization.

Sample Data:

"This is tokenizing."

Character Level

[T] [h] [i] [s] [i] [s] [t] [o] [k] [e] [n] [i] [z] [i] [n] [g] [.]

Word Level

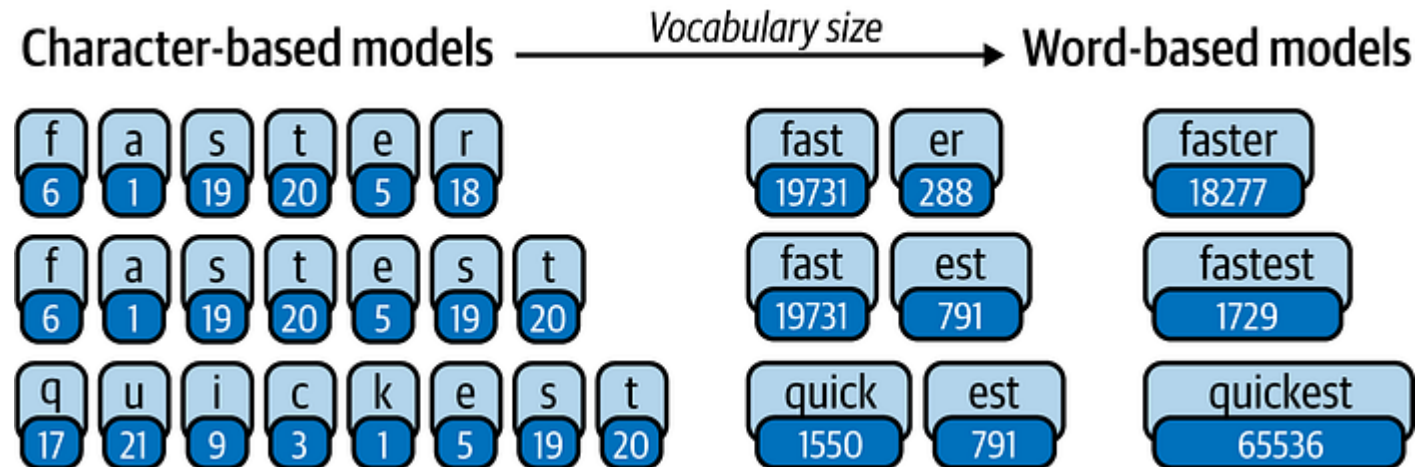
[This] [is] [tokenizing] [.]

Subword Level

[This] [is] [token] [izing] [.]

Subword Tokenization

- On the one hand, we want to **split rare words** into **smaller units** to allow the model to deal with **complex words** and **misspellings**.
- On the other hand, we want to keep **frequent words** as unique entities so that we can **keep** the **length** of our **inputs** to a manageable **size**.



NLTK

- [NLTK](#) (Natural Language Toolkit) is an open-source Python library for Natural Language Processing. It has easy-to-use interfaces for over 50 corpora and lexical resources such as WordNet, along with a set of text processing libraries for classification, tokenization, stemming, and tagging.

```
import nltk
from nltk.tokenize import (word_tokenize,
                           sent_tokenize,
                           TreebankWordTokenizer,
                           wordpunct_tokenize,
                           TweetTokenizer,
                           MWETokenizer)

text="Hope, is the only thing stronger than fear! #Hope #Amal.M"
```

```
print(word_tokenize(text))
```

```
['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', '#', 'Hope', '#', 'Amal.M']
```

```
print(sent_tokenize(text))
```

```
['Hope, is the only thing stronger than fear!', '#Hope #Amal.M']
```

```
print(wordpunct_tokenize(text))
```

```
['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', '#', 'Hope', '#', 'Amal', '.', 'M']
```

```
text="What you don't want to be done to yourself, don't do to others..."
tokenizer= TreebankWordTokenizer()
print(tokenizer.tokenize(text))
```

```
['What', 'you', 'do', 'n't', 'want', 'to', 'be', 'done', 'to', 'yourself', ',', 'do', 'n't', 'do', 'to', 'others', '...']
```

```
tweet= "Don't take cryptocurrency advice from people on Twitter 🤔💡"
tokenizer = TweetTokenizer()
print(tokenizer.tokenize(tweet))
```

```
['Don't', 'take', 'cryptocurrency', 'advice', 'from', 'people', 'on', 'Twitter', '🤔', '💡']
```

```
text="Hope, is the only thing stronger than fear! Hunger Games #Hope"
tokenizer = MWETokenizer()
print(tokenizer.tokenize(word_tokenize(text)))
```

```
['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', 'Hunger', 'Games', '#', 'Hope']
```

```
text="Hope, is the only thing stronger than fear! Hunger Games #Hope"
tokenizer = MWETokenizer()
tokenizer.add_mwe(('Hunger', 'Games'))
print(tokenizer.tokenize(word_tokenize(text)))
```

```
['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', 'Hunger_Games', '#', 'Hope']
```

spaCy

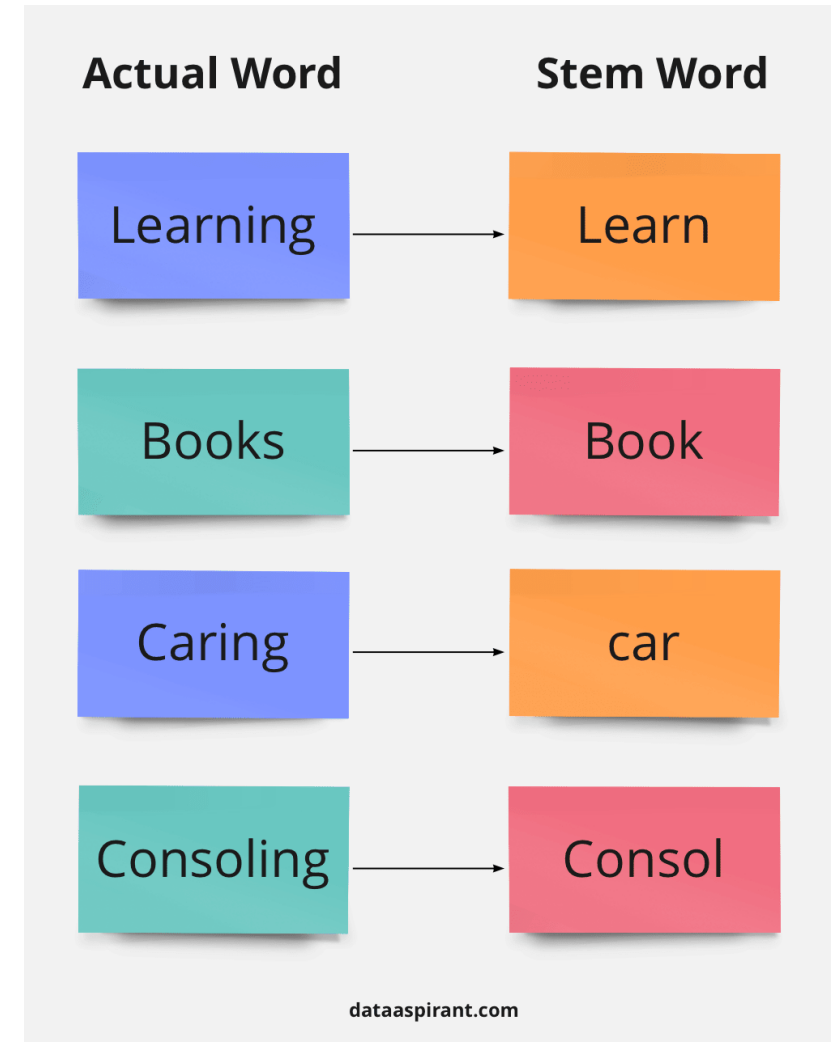
- [SpaCy](#) is an open-source Python library that parses and **understands** large volumes of text. With available models catering to specific languages (English, French, German, etc.), it handles NLP tasks with the most efficient implementation of common algorithms.
- spaCy tokenizer provides the flexibility to specify special tokens that don't need to be segmented, or need to be segmented using special rules for each language, for example punctuation at the end of a sentence should be split off – whereas “U.K.” should remain one token.

NLTK vs. spaCy

- Use spaCy when:** You need fast, accurate, and production-ready NLP tools with pre-trained models for tasks like tokenization, POS tagging, and NER. It's ideal for large-scale applications and real-time processing.
- Use NLTK when:** You are focused on educational purposes, research, or need a wide variety of NLP tools and datasets for prototyping and experimentation. It's valuable for learning and exploring different NLP techniques.

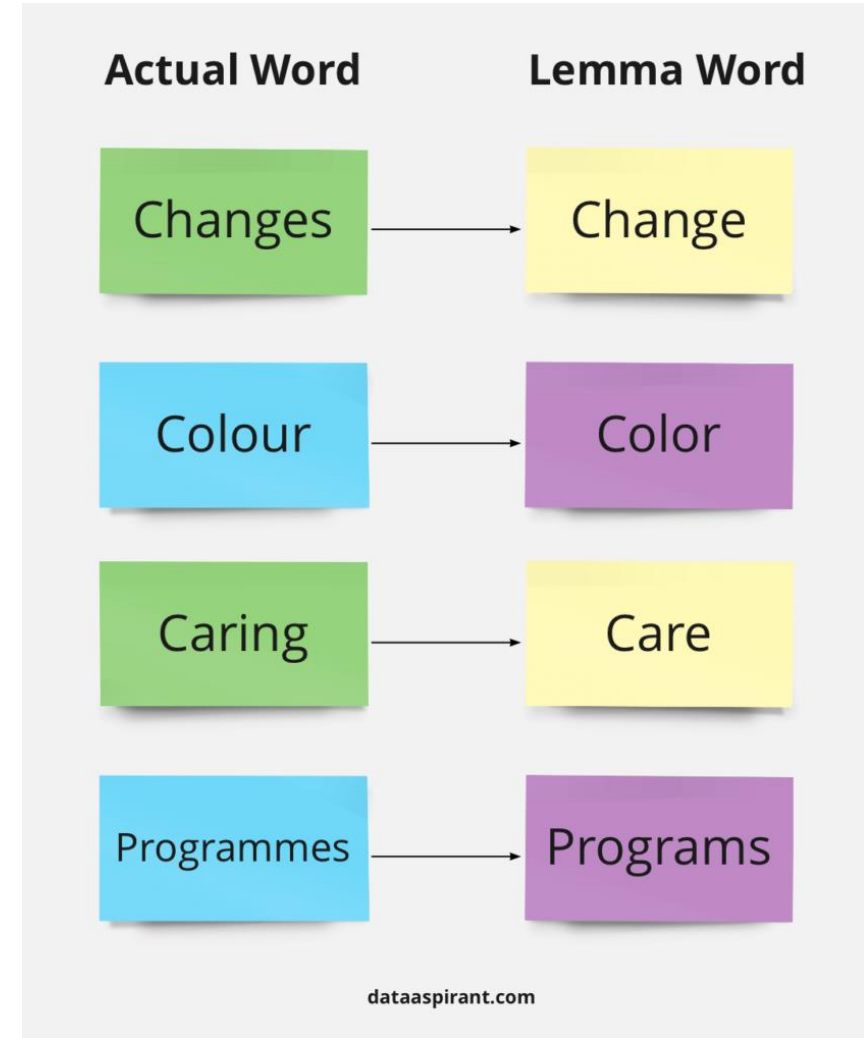
Stemming

- **Definition:** Reducing words to their base or root form by removing suffixes (e.g., “running” to “run”).
- **Algorithms:** Porter Stemmer, Snowball Stemmer.



Lemmatization

- **Definition:** Reducing words to their base form using a dictionary-based approach (e.g., “better” to “good”).
- **Libraries:** WordNet Lemmatizer.



Stemming Vs. Lemmatization

Stemming

- In the process of stemming remove the suffix of words to get a base word.
- Stemming **does not** always provide **meaning** or **dictionary words** as its result.
- The speed of the stemming process is **fast**.

Lemmatization

- Lemmatization follows lexical knowledge to get the root word for original one.
- The resulting words of lemmatization are always **meaningful** and **dictionary words**.
- As compared to stemming the process, the speed of lemmatization is **slow**.

Stopword Removal

- **Definition:** Removing common words (e.g., “the”, “and”) that are often less informative.
- **Purpose:** To reduce noise in text data and improve processing efficiency.

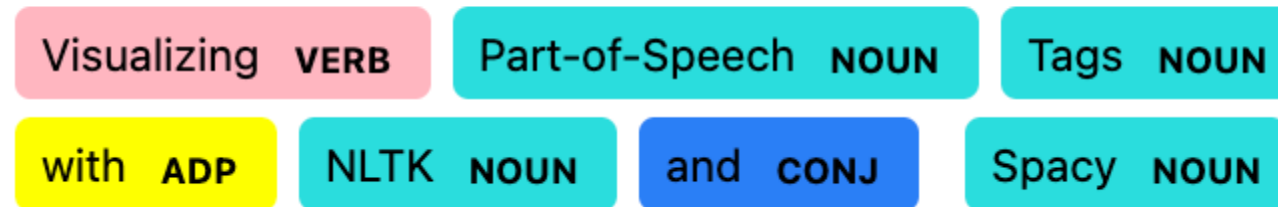
This is an example text for stopwords such as a, an, the etc.

 Fall unders stopwords

dataaspirant.com

Part-of-Speech (POS) Tagging

- **Definition:** Assigning grammatical tags to each word (e.g., noun, verb, adjective).
- **Techniques:** Rule-based taggers, statistical models like Hidden Markov Models (HMMs).

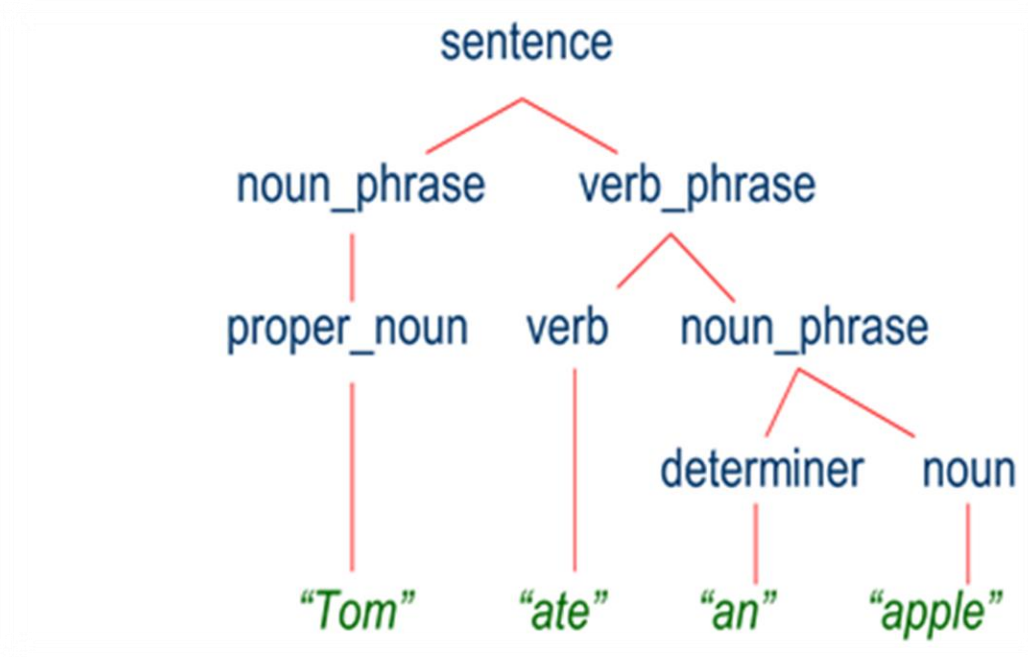


Named Entity Recognition (NER)

- **Definition:** Identifying and classifying entities (e.g., names, dates, locations) within text.
- **Techniques:** Rule-based approaches, machine learning classifiers.

Parsing

- Definition: Analyzing the grammatical structure of sentences.
- Parsing can be more complex than POS tagging because it involves understanding the grammatical structure and relationships between words in a sentence.

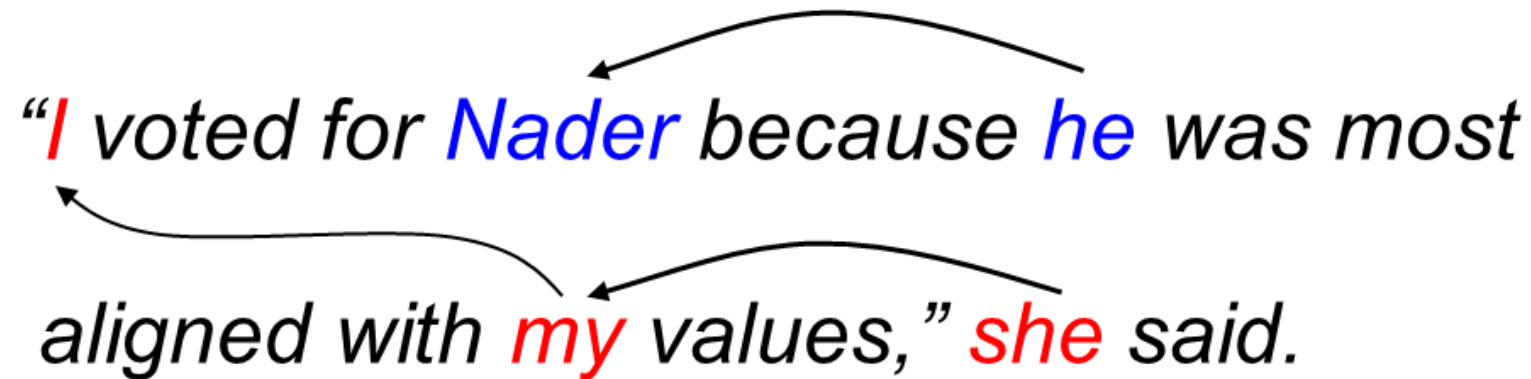


Language Detection

- **Definition:** Identifying the language of the text, is especially useful when dealing with multilingual content.

Coreference Resolution

- Coreference resolution is the task of finding all expressions that refer to the same entity in a text. It is an important step for a lot of higher level NLP tasks that involve natural language understanding such as document summarization, question answering, and information extraction.



*“**I** voted for **Nader** because **he** was most
aligned with **my** values,” **she** said.*

The diagram shows three curved arrows indicating coreference relations: one from 'I' to 'she', one from 'Nader' to 'he', and one from 'my' to 'she'.

Padding & Truncation

- Padding and truncation are key preprocessing steps used to ensure that text sequences are of uniform length, which is essential for many machine learning models and deep learning frameworks.
- **Padding** is the process of adding special tokens to the end (or beginning) of sequences to ensure they are all of the same length. This is particularly useful when working with models that require input sequences of a fixed size.
- **Truncation** involves cutting off parts of sequences that exceed a certain length. This is used to ensure that sequences do not exceed the maximum length allowed by the model or processing pipeline.
- Many models (i.e. RNNs/LSTMs, BERT/Transformers) require input sequences to be of a fixed size. Padding ensures that all sequences are the same length, while truncation ensures sequences are not too long

Truncation and Padding can be combined.

this	sentence	is	short	<PAD>	<PAD>	<PAD>
this	sentence	is	longer	than	the	above
this	sentence	is	the	longest	in	this

E.g. max length = 7